Homework 1: Dynamic Programming in a Gridworld

COSC 527: Reinforcement Learning

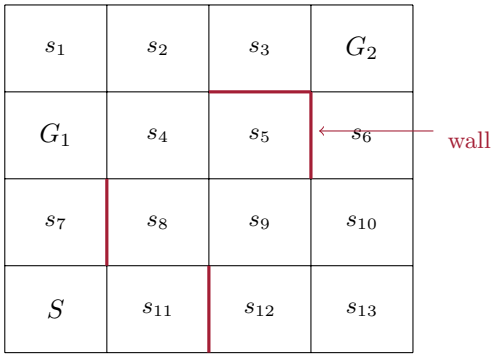
Instructor:	Raffaele Romagnoli
Due date:	Posted on Canvas
Submission:	Source code and a short PDF report

Purpose

In this assignment you will implement dynamic-programming methods for a finite Markov decision process (MDP). You will first evaluate a fixed policy and then use policy improvement, policy iteration, and value iteration to construct an optimal policy. Your implementation should separate the *environment* from the *solution algorithm*, so that the same algorithms can later be used for a different project model.

The deterministic gridworld

The environment contains 16 states arranged as shown below. The agent begins at S. States G_1 and G_2 are terminal goals. Thick red segments are walls *between* states; every square remains a valid state.



For example, the wall between s_5 and s_6 means that moving right from s_5 and moving left from s_6 are blocked. The other walls are interpreted in the same way.

Actions, transitions, and rewards

The action set is

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}.$$

The model is deterministic. A valid move takes the agent to the adjacent state. A move through a wall or outside the grid leaves the agent in its current state. Use the following reward convention:

Transition	Reward R_{t+1}
Enter G_1	$+1$
Enter G_2	$+5$
Cross a wall or leave the grid	-1
Any other valid move	0

After entering a terminal state, the episode ends and its continuation value is zero. A collision with a wall or boundary does *not* terminate the episode.

Required work

1. Environment model

Implement a transition function that accepts (s, a) and returns $(r, s', \texttt{terminal})$. Demonstrate its operation using at least four transitions: an ordinary move, entry into each goal, and a collision. Include these examples in your report.

2. Iterative policy evaluation

Let the initial policy select every action with equal probability:

$$\pi_0(a \mid s) = \frac{1}{4}.$$

Initialize every nonterminal state to $V_0(s) = 0.5$ and terminal states to zero. Using *synchronous* updates, implement

$$V_{k+1}(s) = \sum_a \pi_0(a \mid s) \left[R(s,a) + \gamma V_k\big(s'(s,a)\big) \right].$$

Use $\gamma = 0.9$ and stop when

$$\Delta_k = \max_s |V_{k+1}(s) - V_k(s)| < 10^{-6}.$$

Report V_0, V_1, V_2, the converged V^{π_0}, and the number of sweeps. Display each value function in the same 4×4 arrangement as the maze. Briefly explain why values change even though the policy remains fixed.

3. Policy improvement

From the converged V^{π_0}, calculate

$$q_{\pi_0}(s,a) = R(s,a) + \gamma V^{\pi_0}\big(s'(s,a)\big)$$

for every state and action. Construct a greedy improved policy

$$\pi_1(s) \in \arg\max_a q_{\pi_0}(s,a).$$

Report the four action values and the selected action for s_5, s_6, and one additional state of your choice. If several actions tie, report all greedy actions and state how your program breaks the tie.

4. Policy iteration

Alternate complete policy evaluation and greedy policy improvement until the policy is stable. Report:

- the final policy using arrows in the 4×4 maze;
- the final value function;
- the number of policy-improvement steps; and
- the goal reached from S when the final policy is followed.

5. Value iteration

Implement value iteration using

$$V_{k+1}(s) = \max_a \left[R(s,a) + \gamma V_k\big(s'(s,a)\big) \right].$$

Use the same initialization, discount factor, and stopping tolerance. Compare its final value function and greedy policy with the results from policy iteration. Report the number of sweeps used by each method, but do not assume that sweep counts alone determine which implementation is faster.

6. Experiment: when does the preferred goal change?

Repeat value iteration for

$$\gamma \in \{0.20, 0.40, 0.60, 0.65, 0.70, 0.80, 0.90, 0.99\}.$$

For each value of γ, report $V^*(S)$, the first action selected at S, the terminal goal reached, the number of steps to that goal, and the number of sweeps to convergence.

Then answer the following in your own words:

(a) At approximately what value of γ does the preferred goal or route change? Justify your answer using your computed results.

(b) Why can a larger terminal reward be less attractive when γ is small?

(c) Select one state next to a wall. Why is the value of a collision action not necessarily equal to its immediate reward -1?

(d) Explain the difference between changing V_k during policy evaluation and changing the policy during policy improvement.

Implementation requirements

You may use any programming language. Your program must, however:

- store the MDP model separately from the DP algorithms;
- compute results rather than hard-code tables or policies;
- use a second value table for synchronous policy-evaluation and value-iteration sweeps;
- provide a clear way to change γ, the initial values, and the convergence tolerance; and
- produce readable value tables and policies.

Your code should be sufficiently general that changing the transition and reward model does not require rewriting the DP algorithms.

Submission

Submit one archive containing:

1. all source files and brief instructions for running the program;
2. a PDF report containing the requested tables, policies, comparisons, and written answers; and
3. any output files needed to reproduce the reported results.

Use of Generative AI

Use of generative AI tools is permitted in accordance with the course syllabus. If you use such a tool, include an appendix containing:

- the relevant prompts or questions submitted to the tool;
- a brief description of how the responses influenced your work;
- at least one example of an AI-generated suggestion that you verified, corrected, modified, or rejected; and
- the portions of the submitted code or report for which the tool provided assistance.

A complete transcript is not required unless specifically requested. You remain responsible for understanding, verifying, and explaining every part of your submission. Undisclosed use of generative AI will be handled according to the policies stated in the course syllabus. You are responsible for understanding, verifying, and being able to explain every part of your submitted work.

Evaluation criteria

Component	Weight
Correct environment model	15%
Policy evaluation and reported intermediate values	20%
Policy improvement and policy iteration	20%
Value iteration and comparison	20%
Discount-factor experiment and interpretation	20%
Clarity, reproducibility, and code organization	5%